National University of Computer and Emerging Sciences

Lahore Campus

Data Structures

Assigned Date: 05-11-2026

Due Date: 20-11-2026

Instructor(s)

Mr. Muhammad Naveed

Assignment # 03

Course code: CS2001

Total marks: 120

Questions: 3

Important Instructions:

1. Put your .cpp file and **one PDF for Q1 and Q3** into a single zip folder and upload it onto the Google Classroom submission folder. Name your solution file with your roll number, i.e., Assignment3_24L_1111.zip.
2. You are not allowed to copy solutions from other students. We will check your solution for plagiarism using plagiarism checkers. If any sort of cheating is found, negative marks will be given to all students involved.
3. Late submission of your solution is not allowed.
4. **Q1 (part A) and Q3 must be hand-written and hand-drawn on paper.** Take clear scans of your pages and combine them into one PDF. A typed answer or a drawing made with software will be awarded zero marks for that question.

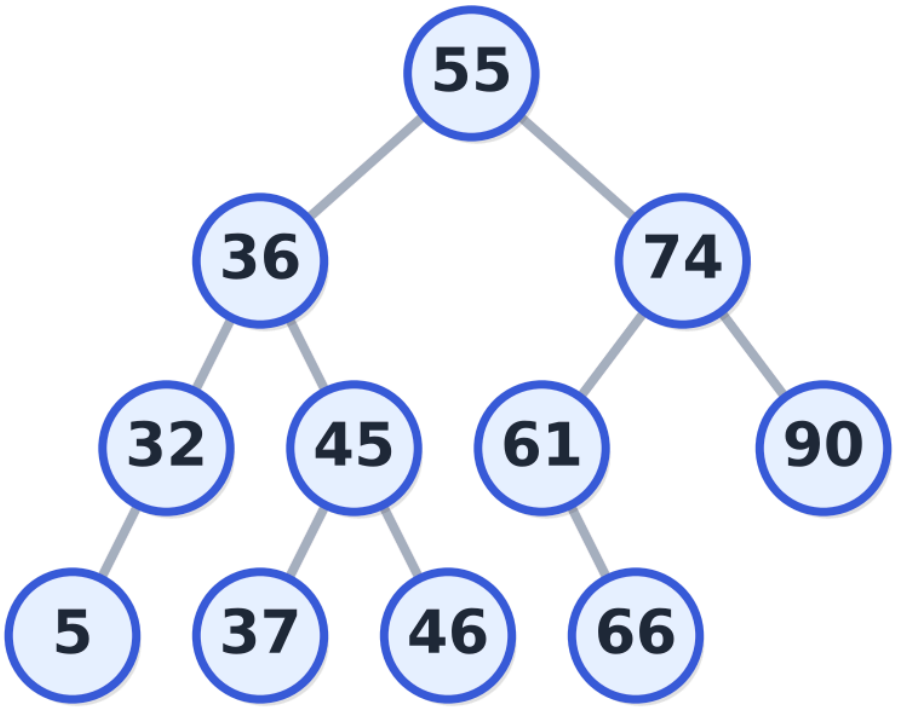
CLO 03: Design appropriate data structures to solve real world problems related to the program.

Implementation Instructions (apply to all questions below)

1. You must design your own custom class/struct for each node. It will contain fields specific to the scenario, not a generic data + next node.
2. You must add your own member functions to solve each part. Function names given below are the minimum required; you may add helper functions.
3. You are not allowed to use built-in containers or algorithms (std::priority_queue, std::vector, std::map, std::unordered_map, std::set, std::list, std::stack, std::queue, std::sort, std::make_heap, std::push_heap, std::pop_heap) to store or process the structure itself. Heap storage must be a raw array (new[]) or raw node pointers that you manage yourself.
4. Where a part states a time requirement (for example O(log n)), a linear scan of the heap will lose the marks for that part.
5. Each question is broken into parts. Read carefully. Submit working, tested code for every part, since later parts build on earlier parts.
6. Also make appropriate print functions for every class that shows all the items in it.

Q1: AVL Trees **[Marks 20+30]:**

The AVL tree below is balanced. Perform the seven operations listed under the figure, **one after the other**, each on the tree left by the previous operation. You have to do it **on paper** and **code** as well. If a node with two children is deleted, replace it with its **inorder successor** (the smallest key in its right subtree).



Part A — On Paper

For every operation, write the BFs (Balance Factors) of the nodes, name the unbalanced node and the rotation (for LR and RL also write the two single rotations in order), and draw the tree after **every** rotation, with the BF written next to each node of the final tree of that operation.

1. Insert 25
2. Insert 43
3. Delete 90
4. Insert 39
5. Insert 35
6. Delete 45
7. Delete 74

Finally, write the **pre-order traversal** and the **height** of the final tree.

Part B — Code

Write a C++ program that performs the same seven operations on an AVL tree that you build yourself. Build the initial tree by inserting the keys **55, 36, 74, 32, 45, 61, 90, 5, 37, 46, 66** in this order (no rotation happens, and the result is exactly the tree in the figure). Your program must have:

- Node and tree design
- insert(key) with rebalancing for all four cases (LL, RR, LR, RL).
- remove(key) using the inorder successor, with rebalancing all the way up to the root (so that cascading rotations work).
- Printing and main() that runs all seven operations in order.

Output format: Print the initial tree. Then, for each operation, print the operation, one line for every rotation performed (in the order they happen, lowest first), and the tree level by level, where every node is printed as key(BF). At the end print the level-by-level traversal and the height. Level-by-level printing needs a queue: **you may use std::queue here (Q1 only)**. Your destructor must free every node.

Example of the format (on a different tree, starting from an empty tree; this is **not** the tree of this question):

```
Insert 30
Level 0: 30(0)

Insert 10
Level 0: 30(1)
Level 1: 10(0)

Insert 20
Rotation LR at 30: left-rotate(10), right-rotate(30)
Level 0: 20(0)
Level 1: 10(0) 30(0)

Insert 40
Level 0: 20(-1)
Level 1: 10(0) 30(-1)
Level 2: 40(0)

Delete 10
Rotation RR at 20: left-rotate(20)
Level 0: 30(0)
Level 1: 20(0) 40(0)

Level-order traversal: 30 20 40
Height of tree: 2
```

Q2: Heaps and Priority Queues **[Marks 50]:**

Scenario: Punjab Emergency Control Room

An emergency control room receives calls about incidents. Waiting incidents are kept on a **board** that holds a limited number of incidents. Ambulance **units** are sent to the incidents. The room must always handle the most urgent incident first, make sure that old incidents do not wait forever (aging), remove cancelled calls, and, when the board is full, drop the least urgent incident to make room for a more urgent one.

Every incident is a node containing:

- incidentID (int, unique, from 1 to 999)
- severity (int, 1 to 5, where 5 is the most critical)
- arrivalTime (int, minute at which the call arrived, 0 or more)
- dueTime (int, the minute by which a unit must reach the scene; it must be after arrivalTime)
- travel (int, minutes a unit needs to reach the scene, more than 0)
- escalations (int, starts at 0, how many levels the incident has been raised)
- heapIndex (int, the current position of this node in the dispatch heap)
- shedIndex (int, the current position of this node in the shed heap, used from Part D)

Every ambulance unit is a node containing:

- unitID (int, positive, unique)
- freeAt (int, the minute at which the unit becomes free)
- heapIndex (int, the current position of this node in the unit heap)

Priority rule: The effective severity of an incident is severity + escalations, and it never goes above 5. Incident X is more urgent than incident Y if:

1. X has a higher effective severity, otherwise
2. X has an earlier dueTime, otherwise
3. X has an earlier arrivalTime, otherwise
4. X has a smaller incidentID.

Because the order depends on four fields and it changes while incidents are waiting, a single integer key will not work. You must write your own comparison.

Board rules:

- The board holds at most capacity incidents. The capacity is given when the control room is created.
- The **dispatch heap** keeps the most urgent incident at the top. The **shed heap** (Part D) keeps the least urgent incident at the top. Both heaps hold pointers to the same incident nodes.

- Every incident remembers its own position in each heap. These positions must be correct after every swap. This is what allows cancel and escalate to work in O(log n).
- Incident IDs are from 1 to 999, so you may find an incident by its ID through an array of node pointers indexed by ID. You may not search the heap array for an ID.

Unit rules: The units are kept in a separate heap. The unit that becomes free earliest is on top. If two units are free at the same minute, the smaller unitID is on top. The control room is created with two numbers: the board capacity and the maximum number of units.

Part A — Classes and Reporting

Design the Incident node, the Unit node, the dispatch heap, the unit heap and the ControlRoom class.

Implement `reportIncident(incidentID, severity, arrivalTime, dueTime, travel)`, which validates the incident and inserts it into the dispatch heap using sift-up. Reject it with an error message, in this order, if: the id is outside 1 to 999 or was already used, the severity is not from 1 to 5, the arrivalTime is negative, the dueTime is not after the arrivalTime, or the travel is 0 or less. A rejected report must leave everything unchanged. (What happens when the board is full is handled in Part D. Until then, assume that the board is never full.)

Implement `printHeap()`, which prints the dispatch heap in array order and then level by level. Each incident is printed as id(S effective severity).

Examples (Control room with capacity 5):

```
reportIncident(101, 3, 0, 40, 8)   -> Incident 101 accepted.
reportIncident(102, 5, 5, 25, 8)   -> Incident 102 accepted.
reportIncident(103, 3, 10, 30, 6)  -> Incident 103 accepted.
reportIncident(104, 3, 12, 30, 9)  -> Incident 104 accepted.
reportIncident(105, 1, 15, 90, 20) -> Incident 105 accepted.
reportIncident(106, 7, 20, 50, 5)  -> ERROR [InvalidIncident]: Incident 106 has severity
7 (must be 1 to 5).
reportIncident(102, 2, 20, 50, 5)  -> ERROR [InvalidIncident]: ID 102 is already used.
reportIncident(107, 2, 20, 20, 5)  -> ERROR [InvalidIncident]: Incident 107 has due time
20, which is not after arrival time 20.
```

```
printHeap():
Heap array: 102(S5) 104(S3) 103(S3) 101(S3) 105(S1)
Level 0: 102(S5)
Level 1: 104(S3), 103(S3)
Level 2: 101(S3), 105(S1)
```

Incident 102 is on top because its severity is highest. The three severity 3 incidents are ordered by dueTime (103 and 104 are due at 30, 101 is due at 40), and 103 beats 104 because it arrived earlier. Incident 103 and 104 are siblings, so either may be on the left.

Part B — Units and Dispatch

Implement `addUnit(unitID, freeAt)`. Reject it with an error message, in this order, if: the unitID is not positive, the unit already exists, freeAt is negative, or the unit pool is full.

Implement `dispatch(now)`, which handles the single most urgent incident on the board:

1. Take the incident `t` on top of the dispatch heap and the unit `u` on top of the unit heap.
2. start = max(now, `u`.freeAt) and eta = start + `t`.travel.
3. If eta is not after `t`.dueTime, the unit is sent. Remove `t` from the board. The unit becomes free again at start + 2 * `t`.travel (it has to come back). Print the dispatch line.
4. Otherwise no unit can reach it in time (`u` is the earliest available unit). Remove `t` from the board, leave the unit unchanged, and report the deadline as missed.

Example (Continuing from Part A):

```
addUnit(1, 0)   -> Unit 1 added (free at 0).
addUnit(2, 18)  -> Unit 2 added (free at 18).
addUnit(1, 5)   -> ERROR [InvalidUnit]: Unit 1 already exists.
addUnit(0, 5)   -> ERROR [InvalidUnit]: Unit ID must be positive.
addUnit(3, -4)  -> ERROR [InvalidUnit]: Unit 3 has free-at time -4 (must be 0 or more).

dispatch(15) -> Dispatched: Unit 1 -> Incident 102 (ETA 23, due 25).
dispatch(20) -> Dispatched: Unit 2 -> Incident 103 (ETA 26, due 30).
dispatch(24) -> ERROR [DeadlineMissed]: Incident 104 cannot be reached by due time 30
(best ETA 40 by Unit 1).
dispatch(24) -> Dispatched: Unit 1 -> Incident 101 (ETA 39, due 40).
```

Only incident 105 is left on the board, and the units are Unit 2 (free at 32) and Unit 1 (free at 47).

Part C — Escalation, Cancellation and Aging

Implement `escalate(incidentID)`, which raises the effective severity of a waiting incident by one level (increase its escalations by one) and puts it in its correct place in the heap. If the incident is not on the board, or its effective severity is already 5, print an error and change nothing.

Implement `cancel(incidentID)`, which removes any waiting incident from the board, not only the top one. **Be careful:** when an element from the middle of a heap is removed and replaced by the last element, that element may need to move up **or** down.

Implement `ageAll(now)`, which handles aging. For every incident on the board, the target is escalations = min((now - arrivalTime) / 30, 5 - severity), using integer division. An incident is escalated only if the target is higher than its current escalations (aging never lowers it).

Visit the incidents in increasing incidentID order and fix each escalated incident's position immediately. It is allowed to scan all incidents in ageAll, but not in escalate or cancel.

Example on a fresh board with the five incidents of Part A:

```
escalate(101) -> Incident 101 escalated to severity 4.
printHeap():
Heap array: 102(S5) 101(S4) 103(S3) 104(S3) 105(S1)

escalate(102) -> ERROR [AlreadyCritical]: Incident 102 is already at severity 5.
escalate(999) -> ERROR [IncidentNotFound]: No incident with ID 999 on the board.
```

Incident 101 was in position 3, below 104 (severity 3). It becomes severity 4, moves above 104, and stops below 102 (severity 5).

Example of aging, again on a fresh board with the five incidents of Part A:

```
ageAll(40) -> Aged to time 40: 2 incident(s) escalated (101 -> S4, 103 -> S4).
printHeap():
Heap array: 102(S5) 101(S4) 103(S4) 104(S3) 105(S1)
```

Working: (40 - 0) / 30 = 1 for incident 101 and (40 - 10) / 30 = 1 for incident 103, so both go from severity 3 to 4. Incident 102 is already at 5. Incident 104 has waited 28 minutes and incident 105 has waited 25 minutes, so neither changes. Incident 101 is visited first and moves above 104. Incident 103 is then visited, but it stays below 102.

Examples of cancel. A new board with capacity 10 and seven incidents. All have severity 3, arrival time 0 and travel 5. Only the due times differ:

```
reportIncident(201, 3, 0, 10, 5)
reportIncident(202, 3, 0, 30, 5)
reportIncident(203, 3, 0, 20, 5)
reportIncident(204, 3, 0, 40, 5)
reportIncident(205, 3, 0, 50, 5)
reportIncident(206, 3, 0, 25, 5)
reportIncident(207, 3, 0, 22, 5)

printHeap():
Heap array: 201(S3) 202(S3) 203(S3) 204(S3) 205(S3) 206(S3) 207(S3)

cancel(204) -> Incident 204 cancelled.
printHeap():
Heap array: 201(S3) 207(S3) 203(S3) 202(S3) 205(S3) 206(S3)

cancel(201) -> Incident 201 cancelled.
printHeap():
Heap array: 203(S3) 207(S3) 206(S3) 202(S3) 205(S3)
```

Working for cancel(204): incident 204 is in position 3. The last incident, 207 (due 22), takes its place. Its parent is 202 (due 30), and 207 is more urgent, so it moves **up**. Working for cancel(201): the root is replaced by the last incident, 206 (due 25). Its children are 207 (due

22) and 203 (due 20), and 203 is the more urgent child and is more urgent than 206, so 206 moves **down**.

Part D — Capacity and Shedding

The board holds at most capacity incidents. You now need quick access to the **least** urgent incident as well, and you may **not** search the heap (not even the leaf half) to find it. Add a second heap, the **shed heap**, over the same incident nodes, with the least urgent incident on top. Every incident now has two positions (heapIndex and shedIndex). Every earlier operation (insert, dispatch, escalate, cancel, ageAll) must keep both heaps correct.

Change `reportIncident` so that, when the incident is valid and the board is full:

- if the new incident is more urgent than the least urgent incident w on the board, remove w from the board (it is **shed**) and insert the new incident;
- otherwise the new incident is **rejected**. It is not stored, and its ID stays free.

Example (Control room with capacity 3):

```
reportIncident(301, 3, 0, 50, 5) -> Incident 301 accepted.
reportIncident(302, 2, 2, 60, 5) -> Incident 302 accepted.
reportIncident(303, 4, 4, 40, 5) -> Incident 303 accepted.
reportIncident(304, 5, 6, 30, 5) -> Board full: Incident 302 (S2) shed to make room.
                                    Incident 304 accepted.
reportIncident(305, 1, 8, 90, 5) -> ERROR [BoardFull]: Incident 305 (S1) is less urgent
than the least urgent waiting incident 301 (S3); not accepted.
reportIncident(306, 3, 10, 45, 5) -> Board full: Incident 301 (S3) shed to make room.
                                    Incident 306 accepted.

printHeap():
Heap array: 304(S5) 303(S4) 306(S3)
Level 0: 304(S5)
Level 1: 303(S4), 306(S3)
```

Incident 306 is more urgent than 301 because both have severity 3 and 306 is due earlier (45 against 50).

Note that escalate raises the importance of an incident, so in the shed heap it moves **away** from the top. Plan for this from Part A.

Part E — Error Handling and Status

Every operation that can fail must print a clear error message in the formats shown in the examples and leave all structures unchanged. The cases are: invalid incident, invalid unit, incident not found, already critical, board empty, no unit, and board full. (A missed deadline is a normal outcome of dispatch, not a failure, because the incident is removed.) An incident that was already dispatched, shed, missed or cancelled is no longer on the board, so escalate or cancel on it gives IncidentNotFound.

```
cancel(555)  -> ERROR [IncidentNotFound]: No incident with ID 555 on the board.
dispatch(5)  on an empty board -> ERROR [NoIncident]: The board is empty; nothing to
dispatch.
dispatch(5)  with incidents but no units -> ERROR [NoUnit]: No ambulance units
available.
```

Check the board first, then the units.

Implement `showStatus()`, which prints the board, the most and least urgent incident, the units and the counters. Units are printed in the order they would be chosen (earliest free first) **without changing the real unit heap**. For example, copy the array into a temporary array and extract from the copy. The counters are: Dispatched (units sent), Missed (deadline missed), Shed, Cancelled, and Rejected (BoardFull rejections only, not invalid reports).

Example 1 (After the dispatches of Part B):

```
showStatus():
===== STATUS =====
Board (1/5): 105(S1)
Most urgent: 105 (S1, due 90) | Least urgent: 105 (S1, due 90)
Units (earliest free first): Unit 2 (free at 32), Unit 1 (free at 47)
Dispatched: 3 | Missed: 1 | Shed: 0 | Cancelled: 0 | Rejected: 0
```

Example 2 (After the example of Part D (no units were added)):

```
showStatus():
===== STATUS =====
Board (3/3): 304(S5) 303(S4) 306(S3)
Most urgent: 304 (S5, due 30) | Least urgent: 306 (S3, due 45)
Units (earliest free first): none
Dispatched: 0 | Missed: 0 | Shed: 2 | Cancelled: 0 | Rejected: 1
```

When the board is empty, print Board (0/N): empty and write none for the most and least urgent incident.

Q3: Huffman Encoding **[Marks 20]:**

Build the Huffman tree for the fourteen symbols in the table below (the first symbol is the space character). Put the smaller node on the left and the larger node on the right, and label every left edge 0 and every right edge 1. If two nodes have the same frequency, the one that existed earlier goes on the left (a symbol is earlier than a merged node, and two symbols are ordered as they appear in the table). Show the final tree with all edges labelled. Then write the code of every symbol. Write the sentence
‘**I LOVE DSA’** in encoded form, and decode the bit string
‘**01110110111110111011111011100100111000011000100111111100110001110100 11’**
into an English sentence.

Symbol/Key	Frequency
Space	22
A	18
C	5
D	8
E	20
H	7
I	15
L	12
O	9
S	10
T	13
U	6
V	4
Y	4